

WardWatch

Municipal Service Delivery Reporting Portal

Project Plan — COMS3009A Software Design 2026 · Wits University

1. Project Overview

WardWatch is a web-based municipal service delivery portal that enables residents to submit, track, and rate service requests (potholes, water outages, electricity faults, waste collection failures) within their ward. Workers manage and update assigned requests; admins oversee the full system. The project is developed by a 6-person Scrum team across 3 sprints using React CRA, Firebase (Firestore, Auth), an Express/Node.js backend, AWS S3, and Azure Static Web Apps.

2. Team & Story Ownership

Member	Role	User Stories	Sprint 1 Story
Sibu	Developer	US001–US004, US006–US009	US001 — Auth (SSO)
Germaine	Developer	US010–US014, US016, US018	US011 — Ward auto-tag
Thendo	Developer	US022–US028	US024 — Update status
Sam	Developer	US029–US034, US036–US037	US034 — Notifications
Kiran	Team Lead / Dev	US038–US045	US041 — Public dashboard
Abdur	Developer	US046–US050	US045 — Volume report

3. Technology Stack

Layer	Technology	Purpose
Frontend	React CRA (JavaScript)	Browser UI — pages, components, routing
Styling	CSS (Navy/Amber design system)	Barlow Condensed + DM Sans fonts
Backend (API)	Node.js + Express (server.js)	REST API: claim, submit, upload, auth
Database	Firebase Firestore	service_requests, users, notifications, messages
Auth	Firebase Auth	Google + Microsoft SSO; role-based access
File storage	AWS S3	Photo uploads for service requests
Geo data	StatsSA GeoJSON	Ward + municipality boundary tagging
Deployment	Azure Static Web Apps	Public URL; serves React build + Node server
CI/CD	GitHub Actions	Push to main → auto-build → Azure deploy
Testing	Jest (50+ tests)	Unit + integration; coverage reports
Version ctrl	GitHub	Feature branches, PRs, commit history

4. Sprint Plan & Milestones

The project runs for 6 weeks with three 2-week sprints.

Sprint	Weeks	Focus	Key Milestone
Sprint 1	Weeks 1–2	Foundation: auth, ward auto-tagging, status update, notifications	Public dashboard, claim, upload, SSO on Azure
Sprint 2	Weeks 3–4	Core features: worker queue, claiming, comments, admin assign	Queue, claim, comment, assign, status change, resident rating

Sprint	Weeks	Focus	Key Milestone
Sprint 3	Weeks 5–6	Polish & completion: feedback, moderation, analytics reports, CSW/BD/Feedback, worker status update, public config	Final Submission: 22 May 2026 23:55

5. High-Level Gantt — Story Distribution by Member

Member / Area	Sprint 1 (Wks 1–2)	Sprint 2 (Wks 3–4)	Sprint 3 (Wks 5–6)
ALL TEAM	Repo setup, CI/CD, Azure config	Sprint 2 planning + retro	Sprint 3 planning + retro
Sibu (Auth)	US001 — Google/Microsoft SSO	US002–004 worker reg, role revoke	US006–009 submission flow
Germaine (Geo)	US011 — StatsSA ward auto-tag	US010, 013, 014, 016 likes + boundaries	US018, US020 history views
Thendo (Mgmt)	US024 — Worker status update	US022, 023, 025 queue+claim+comment	US026, 027, 028 admin assign+close
Sam (Messaging)	US034 — Status notifications	US031, 032 resident-worker chat	US029, 033, 036, 037 categories+rating
Kiran (Dashboard)	US041 — Public Leaflet map dashboard	US039, 040, 042, 043 filters+moderation	US038, 044, 045 feedback+config+analytics
Abdur (Analytics)	US045 — Category volume report	US046, 049 worker performance	US047, 048, 050 custom reports+export

6. Key Milestones & Deadlines

Milestone	Target Date	Owner	Acceptance Criteria
Sprint 1 planning complete	Week 1	Kiran (TL)	Backlog refined; stories estimated; board set up
CI/CD pipeline live	Week 1	All	Push to main deploys to Azure automatically
≥ 4 Sprint 1 stories accepted	End Week 2	All	Passing tests; deployed; demo-able on Azure
Sprint 1 retrospective done	End Week 2	All	Written retrospective submitted per person
Data contracts agreed	Sprint 2 Day 1	All	Field names + types documented before coding
≥ 4 Sprint 2 stories accepted	End Week 4	All	Tests passing; features integrated
Sprint 2 retrospective done	End Week 4	All	Lessons documented; Sprint 3 improvements noted
All Sprint 3 stories accepted	Week 5–6	All	Full app functional on public Azure URL
Screen recording uploaded	22 May 2026	Kiran (TL)	≤6 min; public YouTube/Vimeo link
FINAL SUBMISSION	22 May 2026 23:55 SAST	All	Moodle submission with all artefacts linked

7. Scrum Ceremonies

The team follows Scrum with 3 sprints of ~2 weeks each. The four mandatory ceremony types with required artefacts are:

Ceremony	Frequency	Output Artefact
Sprint Planning	Start of each sprint	Sprint backlog with story points; task assignments
Daily Stand-up	Each working day	Stand-up notes: done / doing / blockers
Backlog Refinement	Mid-sprint	Updated product backlog; re-estimated stories
Sprint Retrospective	End of each sprint	Retrospective report + individual written retrospective (200–400 words)
Sprint Review / Demo	End of each sprint	Demo recording or screenshots; meeting notes

8. Key Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Field name mismatch between teammates	High	High	Define data contracts at Sprint start before any coding
Firebase 431 errors / auth init timing	Medium	Medium	Use <code>onAuthStateChanged</code> ; never assume <code>auth.currentUser</code> is ready
US023 blocks US025 testing	High	Medium	Ensure worker account has assigned requests before testing US025
Azure deployment fails on push	Low	High	CI/CD pipeline tested; secrets stored in GitHub Actions
Leaflet + StatsSA GeoJSON performance	Medium	Low	Lazy-load GeoJSON; limit ward polygons rendered at once
Scope creep / stories not completed	Medium	High	Strict sprint boundary; incomplete stories roll to next sprint

9. Coding Conventions & Repository Rules

Convention	Rule
File naming	snake_case.js — no exceptions (e.g. worker_dashboard.js)
Variables	snake_case for variables and functions; PascalCase for React components
No var	Use const or let only
Firebase keys	Never hardcoded — always <code>process.env.REACT_APP_*</code> from <code>.env</code>
Firebase files	One per actor: <code>worker_firebase.js</code> , <code>admin_firebase.js</code> , <code>resident_firebase.js</code>
Collection name	<code>service_requests</code> (aligned across all teammates)
Branch naming	<code>feature/US###_description</code> e.g. <code>feature/US026_admin_assign</code>
Commit format	US026: add admin request assignment UI
PR process	Branch from main → PR → 1 peer review → merge → auto-deploy
No direct push	Never push directly to main
Environment	<code>.env</code> listed in <code>.gitignore</code> ; never committed